

Содержание

Введение.....	3
Обзор операторов SQL.....	3
Операторы изменения базы данных SQL.....	3
Операции с базой данных.....	4
Операции с таблицами.....	4
CONSTRAINT.....	4
Первичный ключ.....	5
Правила нормализации.....	5
Внешний ключ.....	6
Автоинкремент.....	7
Индексы.....	8
Представления.....	8
Типы данных.....	9
Операторы манипуляции данными.....	9
Оператор вставки.....	9
Оператор обновления.....	9
Оператор удаления.....	10
Использование с SELECT.....	10
SQL-запрос.....	11
Логические шаги выполнения SQL-запроса.....	11
1. Операторы фазы FROM.....	12
Операторы соединения.....	12
2. Операторы фазы WHERE.....	13
Специальные операторы.....	13
3. Операторы фазы GROUP BY.....	14
4. Операторы фазы HAVING.....	14
5. Операторы фазы SELECT.....	15
Операторы множеств.....	15
6. Операторы фазы ORDER BY.....	16
7. Операторы фазы LIMIT/OFFSET.....	16

8. Оператор DISTINCT.....	17
9. Операторы, зависящие от места написания.....	17
Агрегатные функции.....	17
Арифметические операторы.....	18
Побитовые операторы.....	18
Строковые операторы.....	18
Операторы сравнения.....	19
Логические операторы.....	19
Оператор CASE.....	20
Описание:.....	20
Синтаксис:.....	20
Пример использования:.....	21
Приоритет операторов.....	22
Процедуры, триггеры и функции.....	23
Оператор присваивания.....	23
Процедуры.....	23
Триггеры.....	23
Функции.....	24
Ключевые различия.....	24
Заключение.....	24

Введение

SQL (Structured Query Language) — это язык, используемый для управления и манипуляции данными в реляционных базах данных. Этот документ предоставляет подробный обзор операторов SQL, включая их использование, примеры и рекомендации. Здесь вы найдете информацию о различных типах операторов, таких как операторы изменения базы данных, манипуляции данными, фильтрации, группировки, сортировки и многие другие. Цель этого документа — помочь вам лучше понять и эффективно использовать SQL-операторы в вашей работе с базами данных.

Обзор операторов SQL. Операторы изменения базы данных SQL

Операторы изменения базы данных SQL используются для изменения структуры и ограничений базы данных. Это включает создание, изменение и удаление объектов базы данных, а также управление ограничениями и индексами.

Операции с базой данных

Операции с базой данных включают создание, удаление и резервное копирование баз данных.

Оператор	Описание	Пример
CREATE DATABASE	Создает новую базу данных	CREATE DATABASE mydb;
DROP DATABASE	Удаляет существующую базу данных	DROP DATABASE mydb;

Оператор	Описание	Пример
BACKUP DATABASE	Создает резервную копию базы данных	BACKUP DATABASE mydb TO DISK = 'backup.bak';

Операции с таблицами

Операции с таблицами включают создание, удаление и изменение таблиц.

Оператор	Описание	Пример
CREATE TABLE	Создает новую таблицу	CREATE TABLE employees (id INT PRIMARY KEY, name VARCHAR(50), age INT);
DROP TABLE	Удаляет существующую таблицу	DROP TABLE employees;
ALTER TABLE	Изменяет структуру таблицы	ALTER TABLE employees ADD COLUMN salary INT;

CONSTRAINT

Ограничения (**CONSTRAINTs**) — это правила, применяемые к данным в таблицах для обеспечения их целостности. Они могут быть созданы как часть создания таблицы, как в кратком синтаксисе рядом с полем, так и в длинном с использованием ключевого слова **CONSTRAINT**. Они также могут быть созданы с помощью **ALTER TABLE** после первоначального создания таблицы.

Ограничение	Описание	Пример
PRIMARY KEY	Уникально идентифицирует каждую запись в таблице	CREATE TABLE employees (id INT PRIMARY KEY, name VARCHAR(50));
FOREIGN KEY	Связывает две таблицы	ALTER TABLE orders ADD CONSTRAINT fk_customer FOREIGN KEY (customer_id) REFERENCES customers(id);
NOT NULL	Гарантирует, что столбец не может содержать NULL	CREATE TABLE employees (id INT NOT NULL, name VARCHAR(50));
UNIQUE	Гарантирует уникальность всех значений в столбце	CREATE TABLE employees (email VARCHAR(50) UNIQUE);
CHECK	Гарантирует выполнение условия для столбца	CREATE TABLE employees (age INT CHECK (age >= 18));
DEFAULT	Устанавливает значение по умолчанию для столбца	CREATE TABLE employees (status VARCHAR(10) DEFAULT 'active');

Первичный ключ

Первичный ключ (PRIMARY KEY) — это столбец или набор столбцов, которые уникально идентифицируют каждую строку в таблице. Он гарантирует, что в столбце(ах) первичного ключа не будет дублирующихся или NULL значений.

- **Использование:** Первичные ключи определяются при создании таблицы или добавляются позже с помощью оператора **ALTER TABLE**.
- **Хранение:** Первичные ключи автоматически индексируются базой данных, что улучшает производительность запросов.

Правила нормализации

Нормализация — это процесс организации базы данных в серию связанных таблиц для уменьшения избыточности и улучшения целостности данных. Основные правила нормализации:

1. **Первая нормальная форма (1NF):** Убедитесь, что каждый столбец содержит атомарные (неделимые) значения, а каждая строка уникальна.
 - Пример: Избегайте хранения нескольких значений в одном столбце.
2. **Вторая нормальная форма (2NF):** Убедитесь, что все неключевые атрибуты полностью зависят от первичного ключа.
 - Пример: Устраните частичные зависимости, разделив данные на отдельные таблицы.
3. **Третья нормальная форма (3NF):** Убедитесь, что все атрибуты зависят только от первичного ключа, а не от других неключевых атрибутов.
 - Пример: Устраните транзитивные зависимости, создав дополнительные таблицы.

4. Нормальная форма Бойса-Кодда (BCNF): Более строгая версия 3NF, где каждый детерминант является кандидатом на ключ.

- Пример: Устраните аномалии, вызванные пересекающимися кандидатами на ключи.

5. Четвертая нормальная форма (4NF): Убедитесь, что многозначные зависимости устранены.

- Пример: Избегайте хранения несвязанных данных в одной таблице.

6. Пятая нормальная форма (5NF): Убедитесь, что данные разделены на минимально возможные таблицы без потери информации.

- Пример: Разделите таблицы для устранения избыточности, вызванной зависимостями соединений.

Внешний ключ

Внешний ключ (FOREIGN KEY) — это столбец или набор столбцов в одной таблице, которые ссылаются на первичный ключ в другой таблице. Он устанавливает связь между двумя таблицами и обеспечивает ссылочную целостность.

- **Использование:** Внешние ключи определяются при создании таблицы или добавляются позже с помощью оператора **ALTER TABLE**.
- **Хранение:** Внешние ключи хранятся как метаданные в базе данных и не индексируются автоматически (хотя индексация рекомендуется для повышения производительности).
- **Действия при обновлении и удалении:**
 - **ON UPDATE CASCADE:** Обновляет значение внешнего ключа в дочерней таблице при обновлении значения первичного ключа в родительской таблице.
 - **ON DELETE CASCADE:** Удаляет строки в дочерней таблице при удалении соответствующей строки в родительской таблице.

- **ON UPDATE SET NULL / ON DELETE SET NULL:** Устанавливает значение внешнего ключа в **NULL** в дочерней таблице при обновлении или удалении строки в родительской таблице.
- **ON UPDATE RESTRICT / ON DELETE RESTRICT:** Запрещает обновление или удаление строки в родительской таблице, если у нее есть связанные строки в дочерней таблице.
- **ON UPDATE SET DEFAULT / ON DELETE SET DEFAULT:** Устанавливает значение внешнего ключа в значение по умолчанию в дочерней таблице при обновлении или удалении строки в родительской таблице.
- **ON UPDATE NO ACTION / ON DELETE NO ACTION:** Запрещает обновление или удаление строки в родительской таблице, аналогично **RESTRICT**, но проверка откладывается до конца транзакции.

Пример:

```
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    FOREIGN KEY (customer_id) REFERENCES customers(id)
    ON UPDATE CASCADE
    ON DELETE SET NULL
);
```

Автоинкремент

Атрибут **AUTO_INCREMENT** используется для автоматической генерации уникального номера для новых строк.

Оператор	Описание	Пример
AUTO_INCREMENT	Автоматически генерирует уникальные номера	<code>CREATE TABLE employees (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(50));</code>

Индексы

Индексы используются для ускорения извлечения данных из таблицы. Они работают, создавая структуру данных (обычно В-дерево или хеш-таблицу), которая позволяет базе данных быстро находить строки на основе значений индексированных столбцов. Без индекса база данных должна выполнять полное сканирование таблицы для поиска совпадающих строк, что может быть трудоемким для больших таблиц.

Оператор	Описание	Пример
CREATE INDEX	Создает индекс в таблице	<code>CREATE INDEX idx_name ON employees(name);</code>
DROP INDEX	Удаляет индекс из таблицы	<code>DROP INDEX idx_name;</code>

Представления

В SQL представление (VIEW) — это виртуальная таблица, основанная на результирующем наборе SQL-запроса.

Представление содержит строки и столбцы, как и реальная таблица. Поля в представлении — это поля из одной или нескольких реальных таблиц в базе данных.

Вы можете добавлять SQL-запросы и функции в представление и представлять данные так, как если бы они поступали из одной таблицы.

Оператор	Описание	Пример
CREATE VIEW	Создает виртуальную таблицу	<code>CREATE VIEW employee_view AS SELECT name, age FROM employees;</code>
DROP VIEW	Удаляет виртуальную таблицу	<code>DROP VIEW employee_view;</code>

Типы данных

Типы данных SQL определяют тип данных, которые могут быть сохранены в столбце.

Тип данных	Описание	Пример
INT	Целочисленные данные	<code>id INT</code>
VARCHAR	Строка переменной длины	<code>name VARCHAR(50)</code>
DATE	Дата	<code>hire_date DATE</code>
FLOAT	Число с плавающей запятой	<code>salary FLOAT</code>

Операторы манипуляции данными

Операторы манипуляции данными используются для изменения данных в базе данных. Это включает добавление новых записей, обновление существующих записей и удаление записей.

Оператор вставки

Оператор **INSERT** используется для добавления новых строк в таблицу.

Оператор	Описание	Пример
INSERT	Добавляет новые строки в таблицу	INSERT INTO employees (name, age, department) VALUES ('John', 30, 'HR');

Оператор обновления

Оператор **UPDATE** используется для изменения существующих строк в таблице.

Оператор	Описание	Пример
UPDATE	Изменяет существующие строки в таблице	UPDATE employees SET age = 31 WHERE name = 'John';

Оператор удаления

Оператор **DELETE** используется для удаления строк из таблицы.

Оператор	Описание	Пример
DELETE	Удаляет строки из таблицы	DELETE FROM employees WHERE name = 'John';

Использование с SELECT

Операторы манипуляции данными также могут использоваться в сочетании с оператором **SELECT** для извлечения данных перед выполнением операций.

Использование	Описание	Пример
INSERT ... SELECT	Вставляет строки в таблицу на основе запроса SELECT	INSERT INTO employees (name, age) SELECT name, age FROM temp_employees;
UPDATE ... SELECT	Обновляет строки в таблице на основе запроса SELECT	UPDATE employees SET age = (SELECT MAX(age) FROM temp_employees) WHERE department = 'HR';
DELETE ... SELECT	Удаляет строки из таблицы на основе запроса SELECT	DELETE FROM employees WHERE id IN (SELECT id FROM temp_employees);

SQL-запрос

Логические шаги выполнения SQL-запроса

Этот раздел описывает логические шаги выполнения SQL-запроса в порядке их обработки.

Порядок шага	Фаза	Описание
1	FROM	Указывает исходные таблицы или представления, из которых извлекаются данные.
2	WHERE	Фильтрует строки на основе указанных условий до группировки или агрегации.
3	GROUP BY	Группирует строки, имеющие одинаковые значения в указанных столбцах.
4	HAVING	Фильтрует группы, созданные оператором GROUP BY , на основе агрегатных условий.
5	SELECT	Определяет, какие столбцы или выражения

Порядок шага	Фаза	Описание
		включить в результирующий набор.
6	ORDER BY	Сортирует результирующий набор на основе указанных столбцов или выражений.
7	LIMIT/OFFSET	Ограничивает количество возвращаемых строк или пропускает указанное количество строк.
8	DISTINCT	Удаляет дублирующиеся строки из результирующего набора.

1. Операторы фазы FROM

Оператор **FROM** указывает исходные таблицы или представления, из которых извлекаются данные. Это начальная точка выполнения любого SQL-запроса.

Оператор	Описание	Пример
FROM	Указывает исходные таблицы или представления, из которых извлекаются данные	SELECT * FROM employees;

Операторы соединения

Операторы соединения (Join) используются для объединения строк из двух или более таблиц на основе связанного столбца между ними. Эти операторы позволяют гибко выполнять запросы и объединять данные из нескольких таблиц.

Оператор	Описание	Пример
INNER JOIN	Возвращает строки, имеющие совпадающие значения в обеих таблицах	<code>SELECT * FROM table1 INNER JOIN table2 ON table1.id = table2.id;</code>
LEFT JOIN	Возвращает все строки из левой таблицы и совпадающие строки из правой таблицы	<code>SELECT * FROM table1 LEFT JOIN table2 ON table1.id = table2.id;</code>
RIGHT JOIN	Возвращает все строки из правой таблицы и совпадающие строки из левой таблицы	<code>SELECT * FROM table1 RIGHT JOIN table2 ON table1.id = table2.id;</code>
FULL JOIN	Возвращает строки, если есть совпадение в любой из таблиц	<code>SELECT * FROM table1 FULL JOIN table2 ON table1.id = table2.id;</code>
CROSS JOIN	Возвращает декартово произведение двух таблиц	<code>SELECT * FROM table1 CROSS JOIN table2;</code>
SELF JOIN	Соединяет таблицу с самой собой	<code>SELECT a.*, b.* FROM table1 a JOIN table1 b ON a.id = b.parent_id;</code>

2. Операторы фазы WHERE

Операторы фильтрации используются для фильтрации строк на основе определенных условий в запросе.

Оператор	Описание	Пример
WHERE	Фильтрует строки на основе условия	<code>SELECT * FROM employees WHERE age > 30;</code>

Специальные операторы

Специальные операторы предоставляют дополнительную функциональность для конкретных случаев использования.

Оператор	Описание	Пример
IN	Проверяет, существует ли значение в списке	<code>SELECT * FROM table WHERE age IN (25, 30, 35);</code>
NOT IN	Проверяет, что значение не существует в списке	<code>SELECT * FROM table WHERE age NOT IN (25, 30, 35);</code>
BETWEEN	Проверяет, находится ли значение в диапазоне	<code>SELECT * FROM table WHERE age BETWEEN 25 AND 30;</code>
NOT BETWEEN	Проверяет, находится ли значение вне диапазона	<code>SELECT * FROM table WHERE age NOT BETWEEN 25 AND 30;</code>

Оператор	Описание	Пример
IS NULL	Проверяет, является ли значение NULL	SELECT * FROM table WHERE age IS NULL;
IS NOT NULL	Проверяет, не является ли значение NULL	SELECT * FROM table WHERE age IS NOT NULL;
EXISTS	Проверяет, возвращает ли подзапрос какие-либо строки	SELECT * FROM table WHERE EXISTS (SELECT 1 FROM another_table);

3. Операторы фазы GROUP BY

Оператор **GROUP BY** организует строки в группы на основе указанных значений столбцов. Эти группы часто используются для агрегатных вычислений.

Оператор	Описание	Пример
GROUP BY	Группирует строки, имеющие одинаковые значения, в сводные строки	SELECT department, COUNT(*) FROM employees GROUP BY department;

4. Операторы фазы HAVING

Оператор **HAVING** фильтрует группы данных, созданные оператором **GROUP BY**, на основе агрегатных условий.

Оператор	Описание	Пример
HAVING	Фильтрует группы на основе условия	SELECT department, COUNT(*) FROM employees GROUP BY department HAVING COUNT(*) > 5;

5. Операторы фазы SELECT

Оператор	Описание	Пример
SELECT	Определяет, какие столбцы или выражения включить в результирующий набор	SELECT name, age FROM employees;
ALIAS	Временно переименовывает столбец или таблицу на время выполнения запроса	SELECT name AS employee_name FROM employees;

Операторы множеств

Операторы множеств объединяют результаты двух или более запросов.

Оператор	Описание	Пример
UNION	Объединяет результаты двух запросов и удаляет дубликаты	SELECT name FROM table1 UNION SELECT name FROM table2;
UNION ALL	Объединяет результаты двух запросов без удаления дубликатов	SELECT name FROM table1 UNION ALL

Оператор	Описание	Пример
		SELECT name FROM table2;
INTERSECT	Возвращает общие записи из двух запросов	SELECT name FROM table1 INTERSECT SELECT name FROM table2;
EXCEPT	Возвращает записи из первого запроса, которых нет во втором	SELECT name FROM table1 EXCEPT SELECT name FROM table2;

6. Операторы фазы ORDER BY

Оператор **ORDER BY** сортирует результирующий набор на основе указанных столбцов или выражений.

Оператор	Описание	Пример
ORDER BY	Сортирует результирующий набор на основе указанных столбцов или выражений	SELECT * FROM employees ORDER BY age DESC;

7. Операторы фазы LIMIT/OFFSET

Операторы **LIMIT** и **OFFSET** ограничивают количество возвращаемых строк или пропускают указанное количество строк в результирующем наборе.

Оператор	Описание	Пример
LIMIT	Ограничивает количество возвращаемых строк	<code>SELECT * FROM employees LIMIT 10;</code>
OFFSET	Пропускает определенное количество строк	<code>SELECT * FROM employees LIMIT 10 OFFSET 5;</code>

8. Оператор DISTINCT

Оператор **DISTINCT** используется для удаления дублирующихся строк из результирующего набора запроса. Он гарантирует, что возвращаются только уникальные строки.

Оператор	Описание	Пример
DISTINCT	Удаляет дублирующиеся строки из результирующего набора	<code>SELECT DISTINCT department FROM employees;</code>

9. Операторы, зависящие от места написания

Агрегатные функции

Оператор	Описание	Пример
COUNT()	Возвращает количество строк, соответствующих указанному условию	<code>SELECT COUNT(*) FROM table;</code>

Оператор	Описание	Пример
SUM()	Возвращает сумму значений числового столбца	SELECT SUM(salary) FROM employees;
AVG()	Возвращает среднее значение числового столбца	SELECT AVG(salary) FROM employees;
MIN()	Возвращает наименьшее значение в столбце	SELECT MIN(age) FROM employees;
MAX()	Возвращает наибольшее значение в столбце	SELECT MAX(age) FROM employees;

Арифметические операторы

Арифметические операторы используются для выполнения математических операций с числовыми данными.

Оператор	Описание	Пример
+	Сложение	SELECT 5 + 3;
-	Вычитание	SELECT 5 - 3;
*	Умножение	SELECT 5 * 3;
/	Деление	SELECT 5 / 3;
%	Остаток от деления	SELECT 5 % 3;

Побитовые операторы

Побитовые операторы выполняют побитовые операции с целочисленными данными.

Оператор	Описание	Пример
&	Побитовое И	SELECT 5 & 3;
	Побитовое ИЛИ	SELECT 5 3;
^	Побитовое исключающее ИЛИ	SELECT 5 ^ 3;
~	Побитовое НЕ	SELECT ~5;
<<	Сдвиг влево	SELECT 5 << 1;
>>	Сдвиг вправо	SELECT 5 >> 1;

Строковые операторы

Строковые операторы используются для работы с текстовыми данными.

Оператор	Описание	Пример
	Конкатенация	SELECT 'Hello' 'World';
LIKE	Сопоставление с шаблоном	SELECT * FROM table WHERE name LIKE 'A%';
NOT LIKE	Отрицательное сопоставление с шаблоном	SELECT * FROM table WHERE name NOT LIKE 'A%';

Операторы сравнения

Операторы сравнения используются для сравнения двух значений и возвращают логический результат.

Оператор	Описание	Пример
=	Равно	SELECT * FROM table WHERE age = 30;
<> или !=	Не равно	SELECT * FROM table WHERE age <> 30;
>	Больше	SELECT * FROM table WHERE age > 30;
<	Меньше	SELECT * FROM table WHERE age < 30;
>=	Больше или равно	SELECT * FROM table WHERE age >= 30;
<=	Меньше или равно	SELECT * FROM table WHERE age <= 30;

Логические операторы

Логические операторы используются для объединения нескольких условий в запросе.

Оператор	Описание	Пример
AND	Возвращает true, если оба условия истинны	SELECT * FROM table WHERE age > 30 AND salary > 50000;

Оператор	Описание	Пример
OR	Возвращает true, если хотя бы одно условие истинно	SELECT * FROM table WHERE age > 30 OR salary > 50000;
NOT	Инвертирует результат условия	SELECT * FROM table WHERE NOT age > 30;

Оператор CASE

Оператор **CASE** используется для реализации условной логики в SQL-запросах. Он позволяет возвращать разные значения в зависимости от указанных условий.

Описание:

- Оператор **CASE** может использоваться в различных частях SQL-запроса, таких как **SELECT**, **WHERE**, **ORDER BY** или **HAVING**.
- Он последовательно оценивает условия и возвращает первый совпадающий результат. Если ни одно условие не выполнено, выполняется **ELSE** (если он указан).

Синтаксис:

CASE

WHEN condition1 THEN result1

WHEN condition2 THEN result2

...

ELSE default_result

END

Пример использования:

1. В разделе SELECT:

```
SELECT name,  
       CASE  
         WHEN age < 18 THEN 'Minor'  
         WHEN age BETWEEN 18 AND 65 THEN 'Adult'  
         ELSE 'Senior'  
       END AS age_group  
FROM employees;
```

2. В разделе WHERE:

```
SELECT *  
FROM employees  
WHERE CASE  
       WHEN department = 'HR' THEN salary > 50000  
       ELSE salary > 30000  
END;
```

3. В разделе ORDER BY:

```
SELECT name, age  
FROM employees  
ORDER BY CASE  
       WHEN age < 18 THEN 1  
       WHEN age BETWEEN 18 AND 65 THEN 2  
       ELSE 3  
END;
```

Оператор **CASE** является универсальным инструментом для реализации условной логики непосредственно в SQL-запросах, что делает его мощной функцией для динамической обработки данных.

Приоритет операторов

Понимание приоритета операторов SQL имеет решающее значение для написания точных и эффективных запросов. Операторы с более высоким приоритетом оцениваются раньше операторов с более низким приоритетом. Скобки могут использоваться для явного определения порядка выполнения.

Приоритет	Тип оператора	Операторы
1	Скобки	()
2	Агрегатные функции	COUNT(), SUM(), AVG(), MIN(), MAX()
3	Арифметические	*, /, %
4	Арифметические	+, -
5	Побитовые	&, , ^, ~, <<, >>
6	Строковые	, LIKE, NOT LIKE
7	CASE	CASE
8	Сравнительные	=, <>, !=, >, <, >=, <=
9	Логические	NOT
10	Логические	AND
11	Логические	OR

Процедуры, триггеры и функции

SQL предоставляет мощные инструменты, такие как процедуры, триггеры и функции, для автоматизации и расширения операций с базой данных.

Оператор присваивания

Оператор присваивания используются для присвоения значений переменным.

Оператор	Описание	Пример
=	Присваивает значение	SET @var = 5;

Процедуры

Процедуры — это предварительно скомпилированный SQL-код, который может выполняться как единое целое. Они используются для выполнения повторяющихся задач или сложных операций.

Функция	Описание	Пример
CREATE PROCEDURE	Определяет новую процедуру	CREATE PROCEDURE AddEmployee (name VARCHAR(50), age INT) AS BEGIN INSERT INTO employees (name, age) VALUES (name, age); END;
EXECUTE	Выполняет процедуру	EXECUTE AddEmployee 'John', 30;

Триггеры

Триггеры — это специальные процедуры, которые автоматически выполняются в ответ на определенные события в таблице, такие как INSERT, UPDATE или DELETE.

Функция	Описание	Пример
CREATE TRIGGER	Определяет новый триггер	CREATE TRIGGER trg_after_insert AFTER INSERT ON employees FOR EACH ROW BEGIN INSERT INTO audit_log (action) VALUES ('Insert'); END;
DROP TRIGGER	Удаляет существующий триггер	DROP TRIGGER trg_after_insert;

Функции

Функции — это многократно используемый SQL-код, который возвращает одно значение. Они часто используются для вычислений или преобразования данных.

Функция	Описание	Пример
CREATE FUNCTION	Определяет новую функцию	CREATE FUNCTION GetEmployeeAge (emp_id INT) RETURNS INT AS BEGIN RETURN (SELECT age FROM employees WHERE id = emp_id); END;
SELECT	Вызывает функцию	SELECT GetEmployeeAge(1);

Ключевые различия

- **Процедуры:** Выполняют действия, но не возвращают значения.
- **Функции:** Возвращают значение и могут использоваться в SQL-выражениях.
- **Триггеры:** Автоматически выполняются в ответ на события в таблице.

Заключение

Операторы SQL играют ключевую роль в управлении данными и выполнении запросов в реляционных базах данных. Понимание их функциональности и правильное использование позволяет оптимизировать производительность запросов, обеспечивать целостность данных и упрощать работу с большими объемами информации. Надеемся, что этот документ помог вам углубить знания о SQL-операторах и их применении. Используйте полученные знания для создания эффективных и надежных решений в области управления данными.

Список литературы

- 1) James R. Groff, Paul N. Weinberg, Andrew J. Oppel. SQL: The Complete Reference. Third edition. — 2009 г. — 912 с.
- 2) Alan Beaulieu. Learning SQL: Generate, Manipulate, and Retrieve Data. — 2020 г. — 377 с.
- 3) Renee M. P. Teate. SQL for Data Scientists: A Beginner's Guide for Building Datasets for Analysis. — 2021 г. — 288 с.
- 4) Benjamin Nevarez. High Performance SQL Server: The Go Faster Book. — 2016 г. — 228 с.
- 5) Markus Winand. SQL Performance Explained Everything Developers Need to Know about SQL Performance. — 2012 г. — 24 с.
- 6) Avi Silberschatz, Henry F. Korth, S. Sudarshan. Database System Concepts. — 2010 г. — 1376 с.
- 7) Dmitri Korotkevitch. Pro SQL Server Internals. — 2014 г. — 804 с.
- 8) Stephane Faroult. The Art of SQL. — 2006 г. — 370 с.
- 9) Jonathan Gennick. SQL Pocket Guide: A Guide to SQL Usage. — 2010 г. — 204 с.
- 10) Ben Forta. SQL in 10 Minutes, Sams Teach Yourself. — 2012 г. — 288 с.